

Bloque 1: Fundamentos de ORM

Objetivo del bloque

Entender qué problema resuelve ORM, cómo funciona a nivel conceptual y cuál es la relación entre objetos Java y bases de datos relacionales. Este bloque no trata de herramientas, sino de mentalidad.

1. El problema: dos mundos diferentes

Cuando desarrollamos Java trabajamos con **objetos**.

Cuando almacenamos información usamos **bases de datos relacionales** (tablas).

Estos dos mundos son distintos:

Java (orientado a objetos)	Base de datos (relacional)
Clases	Tablas
Objetos	Filas
Atributos	Columnas
Relaciones por referencias	Relaciones por claves

El problema es que Java no sabe guardar directamente objetos en una base de datos. Y una base de datos no entiende objetos, solo tablas.

2. Cómo se resuelve sin ORM (JDBC)

Sin ORM, el desarrollador tiene que hacer la traducción manual.

Ejemplo:

```
Connection conn = DriverManager.getConnection(
    "jdbc:mysql://localhost:3306/tienda", "root", "1234");
```

```
PreparedStatement stmt = conn.prepareStatement(
    "SELECT * FROM usuarios WHERE id = ?");
```

```
stmt.setInt(1, 1);
```

```
ResultSet rs = stmt.executeQuery();
```

```
Usuario u = null;
```

```
if (rs.next()) {  
    u = new Usuario();  
    u.setId(rs.getInt("id"));  
    u.setNombre(rs.getString("nombre"));  
    u.setEmail(rs.getString("email"));  
}
```

Qué está pasando realmente:

1. Se ejecuta SQL manualmente
2. Se obtiene un resultado (ResultSet)
3. Se convierte manualmente cada columna en atributos del objeto

Este proceso se repite constantemente en cualquier aplicación.

3. Problemas del enfoque manual

Trabajar así genera varios problemas importantes:

3.1 Código repetitivo

Siempre hay que escribir:

- consultas SQL
- apertura/cierre de conexiones
- mapeo de datos

3.2 Acoplamiento fuerte con la base de datos

Si cambia la estructura de la tabla, hay que modificar el código Java.

3.3 Errores frecuentes

Errores típicos:

- nombres de columnas incorrectos
- tipos mal convertidos
- olvidos al mapear campos

3.4 Dificultad de mantenimiento

El código crece rápidamente y se vuelve difícil de leer y mantener.

4. ¿Qué es ORM?

ORM (Object-Relational Mapping) es una técnica que automatiza la conversión entre objetos y tablas.

En lugar de hacer la conversión manualmente, se define una relación entre:

- clases Java
- tablas de base de datos

Y el sistema se encarga del resto.

5. Idea fundamental de ORM

ORM se basa en una equivalencia directa:

Concepto en Java	Concepto en BD
Clase	Tabla
Objeto	Fila
Atributo	Columna

Ejemplo:

Clase Java:

```
class Usuario {  
    int id;  
    String nombre;  
    String email;  
}
```

Representación en base de datos:

Tabla: usuarios

- id
- nombre
- email

Cada objeto de la clase representa una fila en la tabla.

6. Qué hace realmente un ORM

Un ORM realiza automáticamente tareas como:

- Generar SQL (INSERT, SELECT, UPDATE, DELETE)
- Convertir resultados en objetos
- Gestionar conexiones con la base de datos
- Sincronizar cambios entre objetos y base de datos

Ejemplo conceptual:

```
Usuario u = new Usuario();  
u.setNombre("Ana");  
u.setEmail("ana@email.com");
```

Con ORM, este objeto puede convertirse automáticamente en:

```
INSERT INTO usuarios (nombre, email) VALUES ('Ana', 'ana@email.com');
```

El desarrollador no escribe ese SQL directamente.

7. Ventajas del uso de ORM

7.1 Abstracción

Se trabaja con objetos en lugar de SQL.

7.2 Reducción de código

Se elimina gran parte del código repetitivo.

7.3 Mantenibilidad

Cambios en el modelo se reflejan de forma más controlada.

7.4 Productividad

El desarrollo es más rápido.

8. Conceptos clave

8.1 Entidad

Una entidad es una clase Java que representa una tabla.

Ejemplo:

```
class Usuario {  
}
```

Representa la tabla "usuarios".

8.2 Atributos

Los atributos de la clase representan columnas.

String nombre;

Se corresponde con una columna en la base de datos.

8.3 Identidad (Primary Key)

Cada entidad necesita un identificador único.

int id;

En la base de datos:

- PRIMARY KEY (id)

Este campo permite distinguir cada registro.

8.4 Persistencia

Persistir significa almacenar un objeto de Java en la base de datos.

Ejemplo conceptual:

```
Usuario u = new Usuario();  
u.setNombre("Carlos");
```

Persistir implica que este objeto se guarda como una fila en la tabla.

9. Desajuste objeto-relacional

Existe un problema conocido como "Object-Relational Impedance Mismatch".

Se refiere a que:

- Los objetos tienen comportamiento y referencias
- Las bases de datos sólo almacenan datos estructurados

Ejemplo de diferencias:

- Herencia en Java vs no existe directamente en tablas
- Objetos complejos vs tablas simples
- Relaciones por referencias vs claves foráneas

ORM intenta reducir este desajuste.

10. Introducción a JPA

JPA (Java Persistence API) es una especificación estándar de Java para trabajar con ORM.

Características:

- Define cómo mapear objetos a tablas
 - Define cómo realizar operaciones de persistencia
 - No es una implementación
-

11. Implementaciones de JPA

Para usar JPA se necesita una implementación:

- Hibernate (la más utilizada)
- EclipseLink

Relación:

- JPA define las reglas
 - Hibernate ejecuta las operaciones
-

12. Idea final del bloque

ORM no elimina la base de datos ni el SQL.

Lo que hace es introducir una capa intermedia que traduce automáticamente entre:

- objetos Java
- estructuras relacionales

El desarrollador trabaja con objetos, pero internamente siguen ejecutándose consultas SQL.

Comprender esta traducción es fundamental para entender correctamente JPA y evitar errores en bloques posteriores.

13. Ejercicio de ejemplo

```
import java.util.ArrayList;
import java.util.List;
```

```
class Usuario {
    int id;
    String nombre;
    String email;

    public Usuario(int id, String nombre, String email) {
        this.id = id;
        this.nombre = nombre;
        this.email = email;
    }
}

public class Main {

    public static void main(String[] args) {

        // "Tabla en memoria"
        List<Usuario> tablaUsuarios = new ArrayList<>();

        // INSERT (simulación persistencia)
        Usuario u1 = new Usuario(1, "Ana", "ana@mail.com");
        tablaUsuarios.add(u1);

        // SELECT por ID
        Usuario resultado = null;

        for (Usuario u : tablaUsuarios) {
            if (u.id == 1) {
                resultado = u;
            }
        }
    }
}
```

```
}

System.out.println(resultado.nombre);

// UPDATE
resultado.nombre = "Ana Modificada";

// DELETE
tablaUsuarios.remove(resultado);
}
}
```


EJERCICIOS

1. Crear modelos Java puros

Crea las clases:

- Usuario (id, nombre, email)
- Producto (id, nombre, precio)

Simular creación de objetos en `main()`.

2. Simulación de tabla en memoria

Crear una lista `List<Usuario>` y simular una “tabla” añadiendo objetos.

3. Simulación de búsqueda por ID

Recorrer la lista de usuarios y encontrar uno por id.

4. Simulación de actualización

Buscar un objeto en lista y modificar un atributo.

5. Simulación de eliminación

Eliminar un objeto de la lista por id.

6. Modelado de relación en memoria

Crear:

- Usuario con `List<Pedido>`
- Pedido con total

Simular agregación.

7. Identificar mapeo objeto-tabla

Para una clase `Empleado`, escribir cómo sería su tabla equivalente.

8. Construir modelo completo de tienda

Clases:

- Cliente
- Pedido
- Producto

Solo estructura y relaciones en Java.

9. Simular persistencia manual

Crear método `guardarUsuario(List<Usuario> lista, Usuario u)`.